

MATERIALS' DATA BASE UPDATION USING DICTIONARIES

Aim:

To write and execute a Python program for updation of a material's data base using a dictionary.

Interactive Development Environment (IDE) Used :

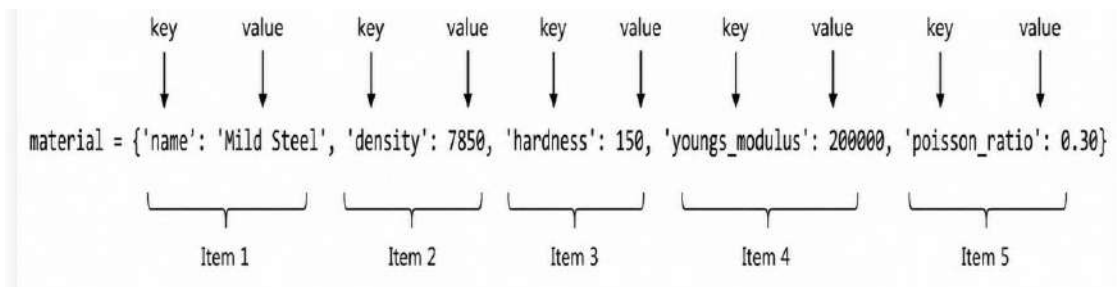
Jupyter NoteBook

Programming Language Used :

Python

Python Concepts used:

A Dictionary is a collection of key-value pairs enclosed within curly brackets ({}). Dictionaries are mutable, allowing values to be added, modified and deleted. In this experiment, a dictionary is used to store the properties of Mild Steel. User-defined functions are used to display the dictionary and to add or delete a key-value pair.



In the given example, `material` is the **dictionary name**, and the dictionary is enclosed within **curly brackets { }**.

- **Key** – Identifies or describes the data. Examples: "name", "density", "hardness".
- **Value** – Stores the actual information associated with the key. Examples: "Mild Steel", 7850, 150.
- **Colon :** – Separates each key from its corresponding value.
- **Comma ,** – Separates one key-value pair from the next.
- **Item** – A complete **key-value pair** is called an item.
- **Curly brackets { }** – Indicate the beginning and end of the dictionary.

Dictionary Operation / Concept	Example Code	Description
Create a Dictionary	<code>material = {"name": "Mild Steel", "density": 7850}</code>	Creates a dictionary containing material properties.
Display Key-Value Pairs	<code>for key, value in material.items(): print(key, ":", value)</code>	Displays each key and its corresponding value.
Access a Value	<code>material["density"]</code>	Returns the value associated with the specified key.
Find Length	<code>len(material)</code>	Returns the number of key-value pairs.
Add a Key-Value Pair	<code>material["yield strength"] = 250</code>	Adds a new key and its corresponding value.
Modify a Value	<code>material["hardness"] = 180</code>	Changes the value of an existing key.
Delete a Key-Value Pair	<code>del material["hardness"]</code>	Deletes the specified key and its value.
Check Key Exists	<code>if "density" in material:</code>	Checks whether the specified key is present.
Get Keys	<code>material.keys()</code>	Returns all keys.
Get Values	<code>material.values()</code>	Returns all values.
Get Items	<code>material.items()</code>	Returns all key-value pairs.
Clear Dictionary	<code>material.clear()</code>	Removes all key-value pairs.

Python Program:

```
# Experiment 5: Material Data Base Using Dictionary

def display(material):
    for key, value in material.items():
        print(key, ":", value)
def add_property(material):
    key = input("Enter property name : ")
    value = input("Enter property value : ")
    material[key] = value
def delete_property(material):
    key = input("Enter property to delete : ")
    if key in material:
        del material[key]
    else:
        print("Property Not Found.")
material = {
    "name": "Mild Steel",
    "density": 7850,
    "hardness": 150,
    "youngs_modulus": 200000,
    "poisson_ratio": 0.30
}
display(material)
choice = int(input("\nEnter your choice (1-Add, 2-Delete): "))
if choice == 1:
    add_property(material)
    display(material)
elif choice == 2:
    delete_property(material)
    display(material)
else:
    print("Invalid Choice.")
```

Sample Input and Output :

Result :

The Python program was written and executed successfully for updation of a material's data base using a dictionary.